

Hardware Vulnerabilities

Andy Baldrian

bristol.ac.uk

Content

- DRAM
- Rowhammer
- Meltdown
- Branch prediction
- Spectre
- Secure boot
- Fault injection



Real hardware is not all 1s and 0s

- It's transistors and capacitors
  - Switching times and moving charge
- Signals on buses
  - with noise and drift
- Things whirring and spinning

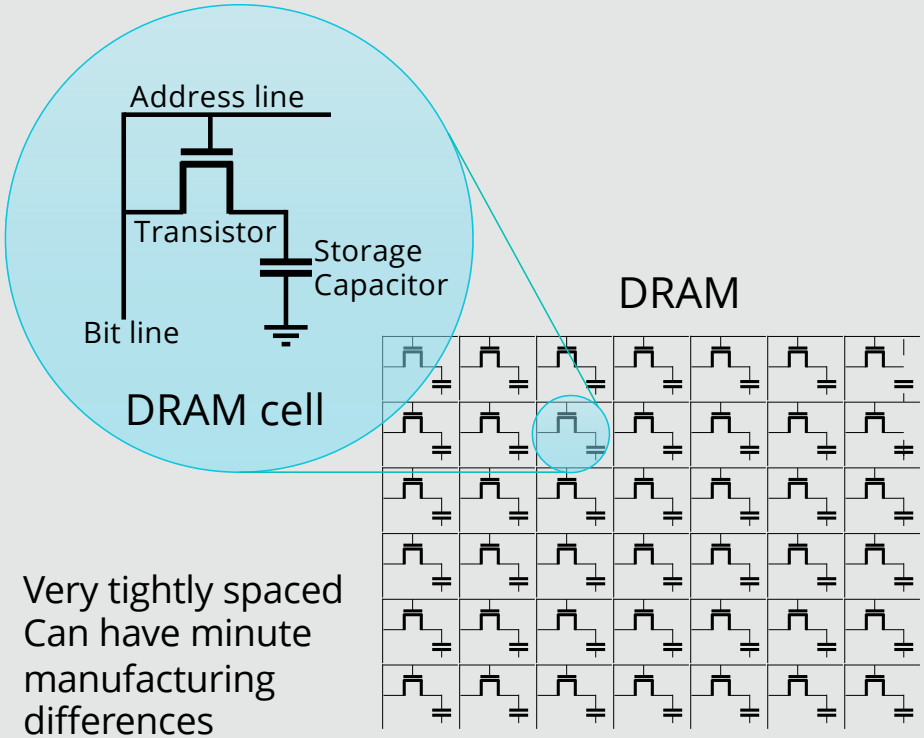


- Give opportunities for attackers:
  - By understanding how the electronics work
  - Reading related signals, side channels
- Fixing issues in hardware costs
  - You cannot trivially fix a silicon wafer
  - You cannot recall old hardware and change the circuits

Rowhammer



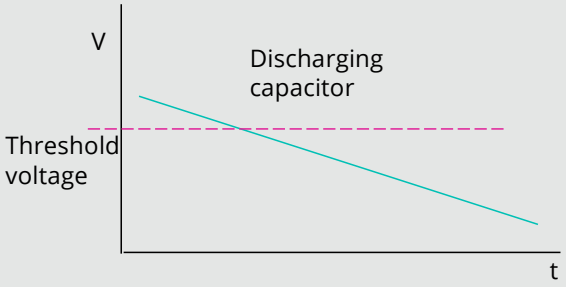
DRAM - Dynamic Random Access Memory



- Very tightly spaced
- Can have minute manufacturing differences

Dynamic Random Access Memory

- Uses a capacitor and a transistor per bit
  - To read the bit value, the charge in the capacitor is discharged
  - The value needs to be put back into the capacitor to restore the bit value
- Capacitors leak charge
  - The 1s and 0s aren't charged or uncharged capacitors
  - Is the current charge more or less than a threshold voltage?



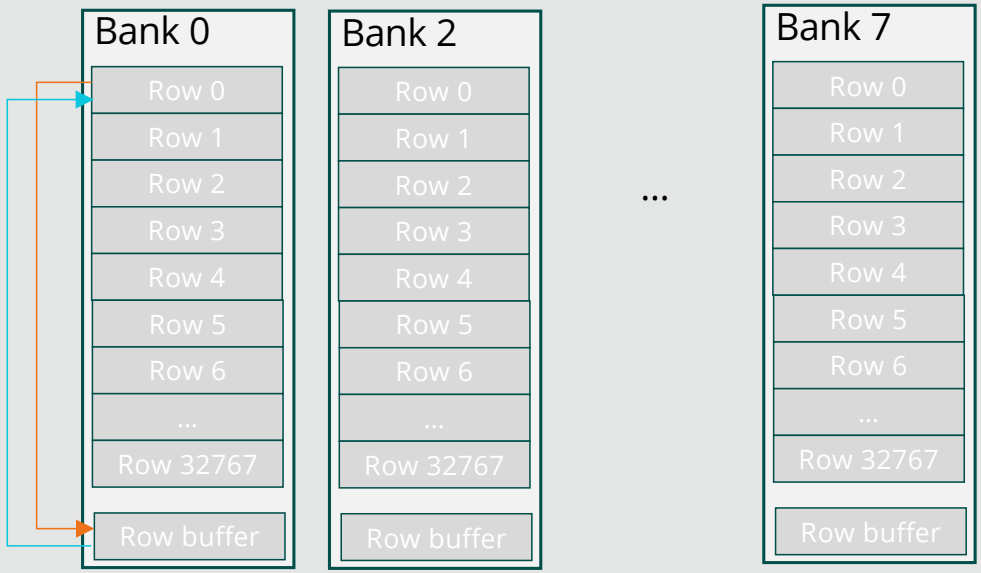
DRAM - Dynamic Random Access Memory

Dynamic Random Access Memory

- Ganged (arranged) into long rows (~8k bits per row)
- Placed into banks of ganged rows

- To read a bit of memory:
  - We find the row it is in
  - Activate the row by letting the capacitors discharge
  - Which copies the row into an active row buffer
  - Rewrite the row before the next row is read

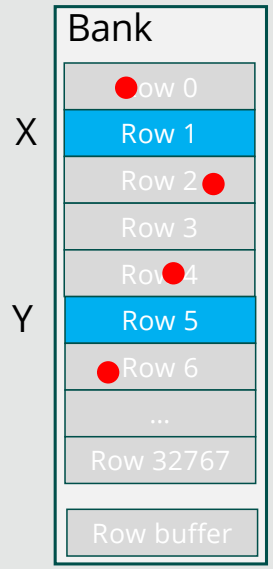
- Current in wires induces current in other nearby wires
- Charge can leak from the capacitor
- A row needs refreshing at least every ~64 ms



Rowhammer


```
code1a:
  mov (X), eax ; Load  value at X into the row buffer
  mov (Y), ebx ; Load  value at Y into the row buffer
  clflush (X)  ; Begin invalidating X's cache line
  clflush (Y)  ; Begin invalidating X's cache line
  mfence       ; Blocks until both clflush complete
               ; In CMU paper, but not actually needed
  jmp code1a   ; Repeat
```

- Read a value at X from memory
  - Move the value into the DRAM row buffer and update after the read.
  - The value will be put into the cache
- Remove the value from the cache
  - The next read will have to go to memory to get the value
- Do this as many times as possible before the next memory refresh



- "Hammering" can cause bits to flip in adjacent rows
- The flip happens within the 64ms before the next refresh
- This is known as single-sided row hammering

Kim, Y., Daly, R., Kim, J., Fallin, C., Lee, J. H., Lee, D., ... & Mutlu, O. (2014).
Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors.

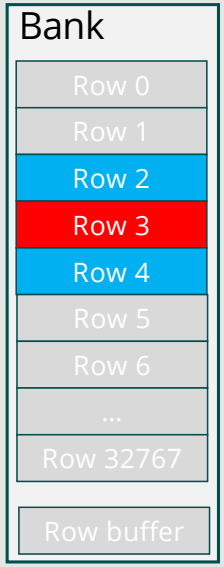
Rowhammer



```
code1a:
  mov (X), eax ; Load  value at X into the row buffer
  mov (Y), ebx ; Load  value at Y into the row buffer
  clflush (X)  ; Begin invalidating X's cache line
  clflush (Y)  ; Begin invalidating X's cache line
  mfence       ; Blocks until both clflush complete
               ; In CMU paper, but not actually needed
  jmp code1a   ; Repeat
```

Bit flipping errors were considered a reliability issue NOT a cyber security issue

- If X and Y are selected so that there is exactly one row between them this row is more likely to accumulate bit flips
- The is known as double-sided row hammering



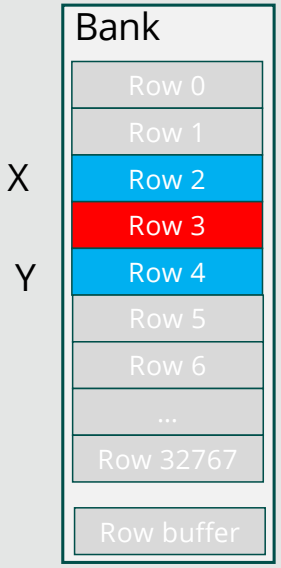
- Manufacturers all knew about the issue
  - Cached memory largely fixes it.
- Several papers discuss it and explore it
- Almost all RAM is vulnerable to it
  - To some extent
- Maybe you could do something malicious theoretically?

Exploiting rowhammer



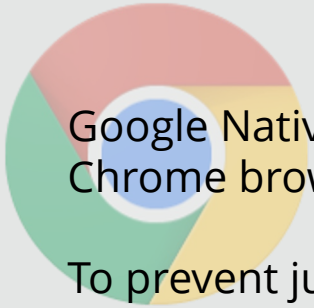
Mark Seaborn & Halvar Flake (and other) continued to explore

- Discover double-sided rowhammer
- Between 30–100% of rows are susceptible to rowhammer
  - Improvements, they later make it 100%.
- Bit flips are consistent
  - Same bits flip every time when you rowhammer the same rows
- Consistent between the same DRAM products
  - If Alice and Bob have the same make of DRAM from the same manufacturer
  - Then if they rowhammer the same rows the same bits will always flip



Mark Seaborn & Halvar Flake (2015) Exploiting the DRAM Rowhammer Bug to Gain Kernel Privileges (Black Hat)

Exploiting rowhammer



Google Native Client (NaCl) was a sandboxing technology used in Google Chrome browsers to safely execute compiled C and C++ code

To prevent jumping into the middle of an x86 instruction
Indirect jumps can only target 32-byte-aligned addresses

Safe instruction sequence:
The address we wish to jump to is in `eax`. This code forces a safe jump

```
and eax , 0xFFFFFFE0 ; Truncate address to 32-bits and
                     ; mask to be 32-byte aligned
add rax , r15        ; Add the sandbox base address
jmp [ rax ]          ; Jump to the loaded code
```

Can we rowhammer this code to escape the sandbox?

Mark Seaborn & Halvar Flake (2015) Exploiting the DRAM Rowhammer Bug to Gain Kernel Privileges (Black Hat)

Exploiting rowhammer



x86 is a dense instruction set

- Different instructions have different lengths
- Some have multiple lengths

Code hiding in plain sight

- Existing technique for exploiting non-aligned jumps

```
48b80f05eb0cf4f4f4f4: mov rax, 0xF4F4F4F40CEB050F
```

What if the bit flip allows you to jump to here?

```
0f 05 syscall ; Do something you shouldn't
eb 0c jmp 0F  ; Jump to next hidden instruction
F4    hlt     ; Padding
```

If we can do a non-aligned jump, we can escape the sandbox

Mark Seaborn & Halvar Flake (2015) Exploiting the DRAM Rowhammer Bug to Gain Kernel Privileges (Black Hat)

Exploiting rowhammer



If we rowhammer this instruction:

```
and eax , 0xFFFFFFE0 ; Truncate address to 32-bits and
```

Can we change it to:

```
and ebx , 0xFFFFFFE0
```

13% of possible bit flips result in exploitable code

Method

- Code section is readable, but not writeable
- Load a sequence of safe code that is validated by NaCl, but is unsafe if we can bit flip the safe jump.
- Make lot of copies of the code, so that we have different addresses to attempt the rowhammer.
- Rowhammer the safe jump code
- Find one that is exploitable and run it.

Mark Seaborn & Halvar Flake (2015) Exploiting the DRAM Rowhammer Bug to Gain Kernel Privileges (Black Hat)

Exploiting rowhammer



Mark Seaborn & Halvar Flake

- Sandbox escape exploit
- Arbitrary code execution
- Linux root privilege escalation

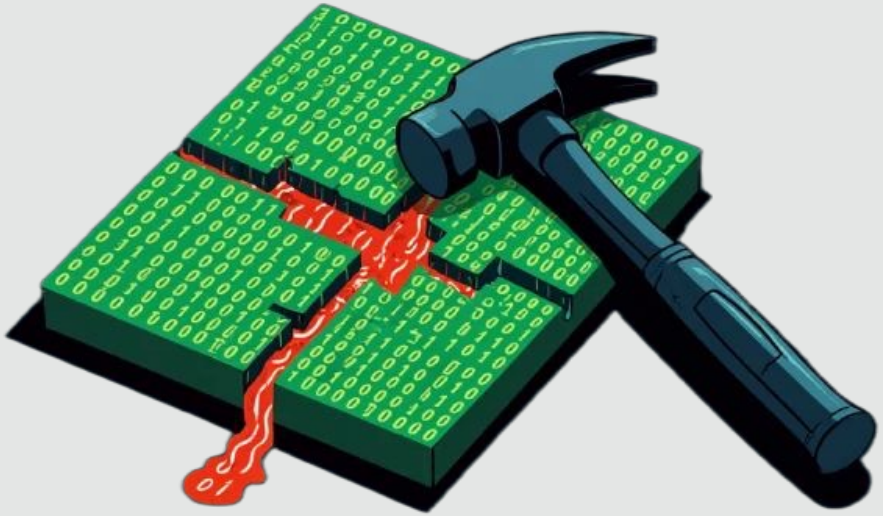
Bit flipping errors are a cyber security issue

Mark Seaborn & Halvar Flake (2015) Exploiting the DRAM Rowhammer Bug to Gain Kernel Privileges (Black Hat)

Rowhammer Mitigations

- Disable the use of `clflush` for user code
- Error Correcting Codes (ECC)
  - One bit flip can be corrected
  - Two, error detected
  - Three, not detected
- Target Row Refresh (TRR)
  - Refresh rows near rows that are read frequently
- Higher DRAM refresh rates

This all costs more money!



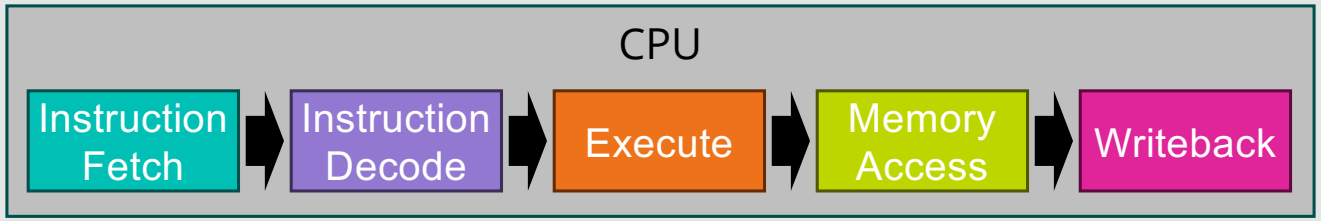
Meltdown & Spectre



Meltdown & Spectre



CPU Pipeline



Instructions:

- Are executed out of order
- Wait until their dependencies are ready
- Retired in order
- Exceptions are checked during retirement

Side-channel attacks



Power consumption



Execution times



CPU cache

Meltdown

Speculative Execution

```
*(volatile char*) 0; // raise exception
array[84 * 4096] = 0;
```

The instruction
`array[84 * 4096] = 0`
was executed and rolled back

But the caches are a separate subsystem and managed by the MMU.

When the second line executes, the page of memory containing `array[84*4096]` will be cached

We can measure the time to read the values in the array `y = array[n * 4096]`



We can detect the index value used before

Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., & Hamburg, M. (2018). Meltdown. *ArXiv*. https://arxiv.org/abs/1801.01207

Meltdown



```
char data *(char*) 0xffffffff81a000e0;
array[data*4096] = 0
```

Method

- The attacker attempts to access memory they do not have access to
- Use the accessed value
  - Speculative execution will mean that the value of `data` will be used to index the `array`
- Manage the exception
  - Use signal handler or other mechanisms to handle the exception
- Read the cache changes and recover the value of `data`

We can read the content of any memory location!

Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., & Hamburg, M. (2018). Meltdown. *ArXiv*. https://arxiv.org/abs/1801.01207

Branch prediction



```
unsigned long factorial(unsigned long n) {
  unsigned long result = 1;
  while(n)          // likely
    result *= n--;
  return result;
}
```

Dynamic branch prediction:

- Branch Target Buffer (BTB)
  - "where does this branch go?"
  - Jumps that go back are more likely
- Branch History Table (BHT)
  - Saturating counter (2 or 4-bits)
  - Counter is incremented when branch is taken or
  - Decremented, branch is not taken

Speculative Execution:

- Assumes a direction of the branch, proceeds to process subsequent instructions
- Pipeline Flush
  If the prediction is wrong, discard the incorrectly executed instructions
- Performance improvements from removing the pipeline stall are bigger than the single pipeline flush
- But there are footprints left in the cache

Spectre


```
if(x < array1_size ) { // likely
  y = array2[array1[x]];
}
```



`y` gets the value from `array2` indexed by whatever is in `array1[x]`

What if `x` is greater than `array1_size`?

The branch prediction has speculatively executed the next instruction, as `x < array1_size` is likely.

- This would segfault, on an instruction you were never going to execute.

As soon as the branch misprediction is detected, the CPU starts the rollback

- Undo changes to registers
- Reset exception flags
- Cancel any memory writes

Kocher, Paul, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg et al. "Spectre attacks: Exploiting speculative execution." *Communications of the ACM* 63, no. 7 (2020): 93-101.

Spectre



```
if(x < array1_size ) { // likely
  y = array2[array1[x]];
}
```

But there are footprints left in the cache

When the second line executes the page of memory containing `array2[array1[x]]` will be cached in preparation for the load into `y`

- A segfault exception is signalled
- Which will never actually happen because the `if` will turn out to be a branch misprediction

Kocher, Paul, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg et al. "Spectre attacks: Exploiting speculative execution." *Communications of the ACM* 63, no. 7 (2020): 93-101.

Spectre



Method

- Train the branch predictor that `x < array1_size` is likely
- Put the cache into a known state.
- Select a value of `x`, so the `array1[x]` reads memory we should not have access to.
- Force the load of a difference cache page for each value of `array1[x]`

```
if(x < array1_size ) { // likely
  y = array2[array1[x]*4096];
}
```

We can then deduce the value at `x`, from the cache hit timing.

We can read the content of any memory location, without generating an exception!

Kocher, Paul, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg et al. "Spectre attacks: Exploiting speculative execution." *Communications of the ACM* 63, no. 7 (2020): 93-101.

Spectre

This attack also worked in JavaScript:

```
if ( index < simpleByteArray.length ) {
  index = simpleByteArray[index | 0];
  index = (((index*4096)|0) & (32*1024*1024-1))|0;
  localJunk ^= probeTable[index| 0] | 0;
}
```



You can leak a byte of memory

- Given a few hours you could leak all of memory on any system where you can host a webpage
- On any cloud service!

Meltdown & Spectre mitigations



The Spectre vulnerability is part of the Meltdown family of attacks:
Meltdown (**CVE-2017-5754**) melts down security barriers
Spectre (**CVE-2017-5753, CVE-2017-5715**) makes speculative execution scary

Affects:

- All operating systems
- All CPUs with branch prediction

Meltdown mitigation

- Kernel Page Table Isolation (KPTI), which isolates kernel memory from user processes.

Spectre is harder to mitigate

- Several different variants
- Indirect Branch Restricted Speculation (IBRS)
- Single Thread Indirect Branch Predictors (STIBP)

- Early patches caused significant slowdowns
- Newer CPUs with hardware mitigations experience negligible performance loss

Current Status: As of 2026, most major variants are patched, but researchers continue to discover new side-channel attacks (e.g., **CVE-2024-2193**)

Fault injection

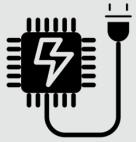


Fault injection

Fault injection is a physical attack technique where an adversary deliberately induces errors or "faults" in a system's hardware to bypass security measures or extract sensitive data.



Clock



Voltage



Electromagnetic



Laser

Voltage manipulation

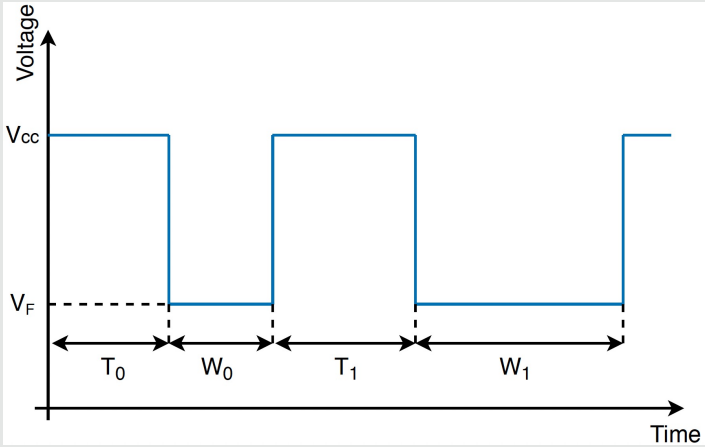
- High-frequency voltage spikes or dips into the device's power rail

Propagation delay

- When the supply voltage drops, the internal transistors operate less efficiently

Data corruption

- Incorrect or incomplete data, leading to instruction skipping or bit flips

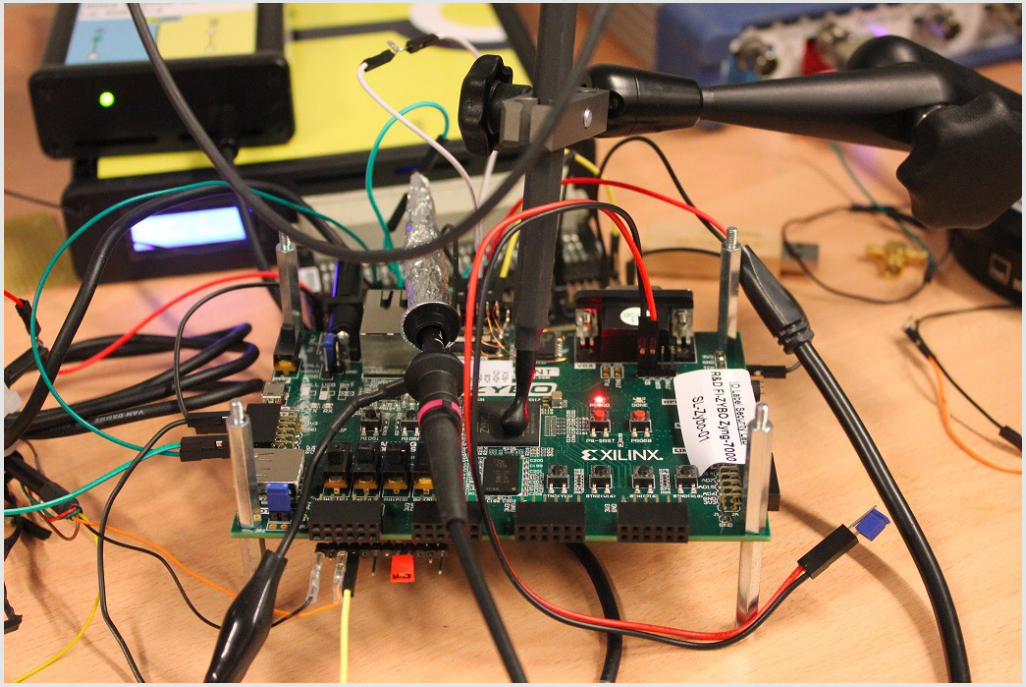


Bar-El, H., Choukri, H., Naccache, D., Tunstall, M., & Whelan, C. (2006).
The sorcerer's apprentice guide to fault attacks.

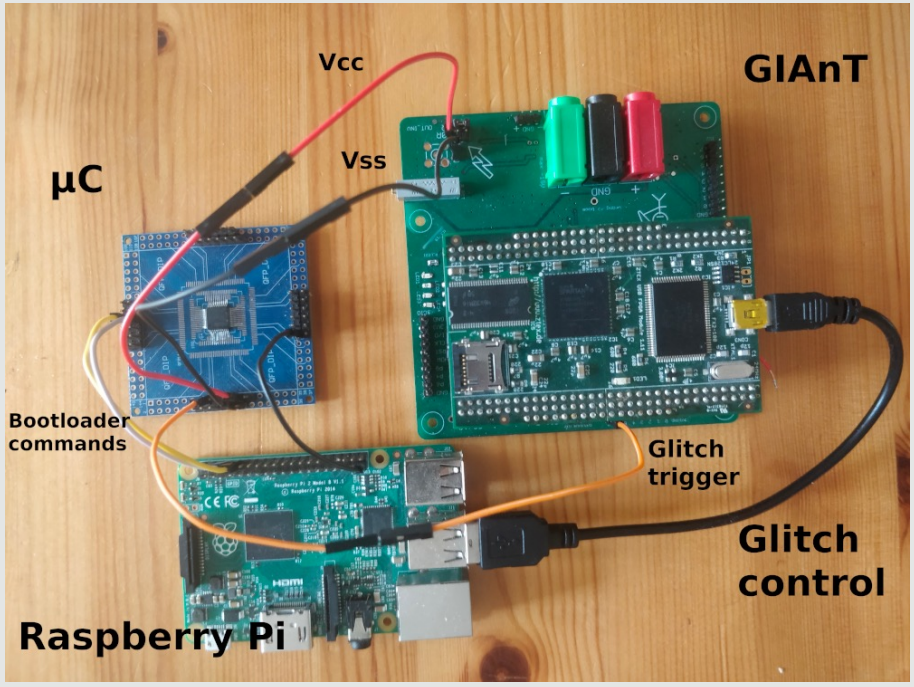
Fault injection , secure boot bypass

Bristol Cyber Security Group

Niek Timmers & Albert Spruyt (2016) Bypassing Secure Boot using Fault Injection, BlackHat Europe



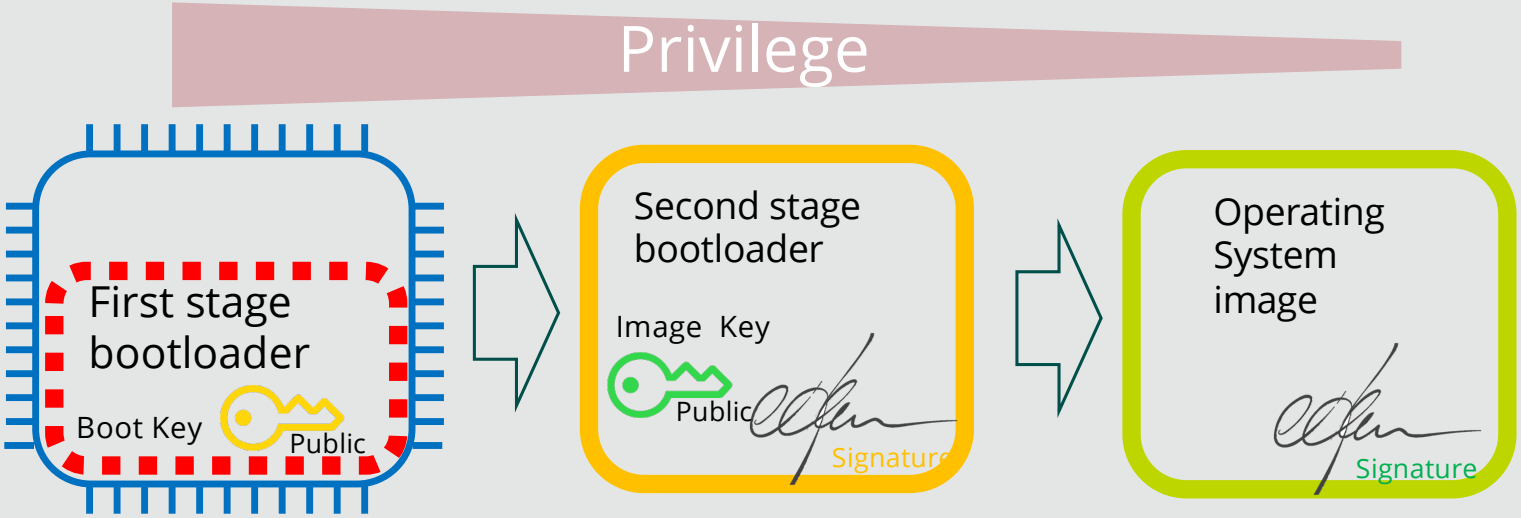
Van den Herrewegen, J., Oswald, D., Garcia, F., & Temeiza, Q. (2020). Fill your boots: Enhanced embedded bootloader exploits via fault injection and binary analysis.



Secure Boot

Booting is the act of initialising the hardware and loading the right software.

Often in two stages.



The first stage bootloader runs in a secure environment, such as:

- Trusted Execution Environment (TEE)
- Loaded from an immutable store

The second stage bootloader may also run in the secured environment.

- The bootloader uses a similar process to validate the OS image before loading it.
- The second stage bootloader can be updated

Fault injection, secure boot bypass



The secure boot code looks something like this:

```
if(mbedtls_pk_verify(&k, SHA256, h, hs, s, ss)) {
  /* Do not boot up the image */
  while(1);
} else {
  /* Boot up the image */
  boot();
}
```

This is what they found:

```
000848 BL      mbedtls_pk_verify ; if
00084C CMP     R0, #0
000850 BEQ     loc_858
000854 loc_854                   ; while(1)
000854 B       loc_854
000858 loc_858                   ; else
000858 SUB     R0, R1, #0x200    ;   log ..
00085C LDR     R0, =(aS_1+4)
000860 BL      printf
                                 ;  continue
                                 ;  to boot
```

If the `mbedtls_pk_verify` fails, enter the loop

- No timing needed

Glitch the Vcc line

- If the branch operation fails, the code will drop to the boot

Fault injection mitigations



Hardware countermeasures

- Detect the glitch or fault

Software countermeasures

- Lower the probability of a successful fault

Software countermeasures do not address the root cause

- Doing the test twice lowers the chance of success
- Use a `volatile` pointer to stop the compiler optimising the checks away

```
unsigned int compare(uint8_t *pwd, size_t pwdlen,
                     uint8_t *salt, size_t saltlen) {
  volatile uint8_t *hash = s_hash; // use volatile pointer

  …

  // Call Argon2i hash function
  argon2i_hash_raw(t_cost, m_cost, parallelism,
                   pwd, pwdlen, salt, saltlen,
                   hash, HASHLEN);
  if(memcmp(s_pwdhash, hash, HASHLEN) == 0) { // <-- 1st
    if(memcmp(s_pwdhash, hash, HASHLEN) == 0) { // <-- 2nd
      return TRUE;
    }
  }
  return FALSE;
}
```

Conclusion



From a security perspective computer hardware is fundamentally broken

- RAM doesn't work
- CPUs are fundamentally broken

Fixing these issues is costly and has performance implications.

Questions?



